

Project Proposal

TechSupport RAG — Multi-Document & Image Troubleshooting Assistant

1. Problem Statement

Technical troubleshooting information is commonly scattered across user manuals, installation guides, troubleshooting PDFs, FAQs, error screenshots, diagrams, and status-indicator images. Users often have to search these sources manually to determine what an error means, identify possible causes, find the correct settings, and follow the appropriate repair steps. This process is slow and difficult for non-technical users. **TechSupport RAG** addresses this problem by creating a unified knowledge base that combines textual documents and visual troubleshooting information and makes the information searchable through natural-language questions.

2. Proposed Solution

The project will implement a **Naïve Retrieval-Augmented Generation (RAG)** pipeline for technical support. Multiple PDF files will be loaded and divided into meaningful text chunks. PNG/JPG troubleshooting images will be analyzed using a vision-capable LLM, which will convert visible error messages, labels, diagrams, indicators, and other useful visual information into searchable descriptions. Both document text and image-derived content will be embedded using **OpenAI text-embedding-3-small** and stored in **ChromaDB**. For every user query, the system retrieves the most relevant knowledge and passes it to an LLM, which generates a concise, grounded troubleshooting response. If sufficient information is not found, the assistant explicitly states that the knowledge base does not contain enough information instead of inventing a solution.

3. Example Scenario — Laptop Troubleshooting

Knowledge Source	Example Content
laptop_manual.pdf	Specifications, controls, charging indicators, ports and operating instructions.
installation_guide.pdf	Initial setup, power connection, display and peripheral configuration.
troubleshooting.pdf	Common hardware/software problems and recommended solutions.
maintenance_guide.pdf	Cleaning, ventilation, battery care and preventive maintenance.
faq.pdf	Frequently asked questions and concise support instructions.
error_screen.png / boot_error.png	Error codes, warning messages and boot-failure screens.
indicator_lights.png	LED colors, blinking patterns and their meanings.
port_diagram.png	Laptop ports and their supported functions.

4. Project Objectives

- Accept multiple PDF documents as a unified technical knowledge base.
- Accept PNG/JPG images containing error messages, diagrams, indicators, screenshots, and labels.
- Extract and normalize text from PDFs using pypdf.
- Analyze images with a vision-capable LLM and convert useful visual information into searchable text.
- Split document text into retrieval-friendly chunks using RecursiveCharacterTextSplitter.
- Generate embeddings and store all knowledge in ChromaDB.
- Retrieve relevant text and image-derived knowledge for each natural-language query.
- Generate concise, context-grounded troubleshooting instructions with GPT-4o-mini.
- Provide references to the source document/image used for the answer.
- Detect insufficient context and clearly inform the user when the knowledge base cannot answer the question.

5. System Workflow

Knowledge Ingestion: PDFs → text extraction → cleaning → chunking → embeddings → ChromaDB. Images → vision analysis → structured description/text → embeddings → ChromaDB.

Query Processing: User question → query embedding → similarity retrieval → relevant document/image context → LLM prompt → grounded troubleshooting response → source reference.

6. Sample User Queries

- My laptop is not charging. What should I check first?
- What does this red indicator mean?

- I am getting this error message. How can I fix it?
- Which port should I use to connect an external monitor?
- My laptop is overheating. What troubleshooting steps are recommended?
- The laptop does not boot after pressing the power button. What should I do?

7. Technology Stack

Area	Technology	Purpose
Programming	Python	Core application and RAG pipeline.
Document Processing	pypdf, RecursiveCharacterTextSplitter	PDF extraction and text chunking.
Image Understanding	Vision-capable LLM	Extract meaningful information from screenshots and diagrams.
Embeddings	OpenAI text-embedding-3-small	Convert text/image descriptions into vectors.
Vector Database	ChromaDB	Store and retrieve embedded knowledge.
LLM	GPT-4o-mini	Generate grounded troubleshooting answers.
Framework / UI	LangChain / Streamlit	Pipeline orchestration and optional web interface.

8. Project Structure

```
TechSupport-RAG/
├── app.py
├── requirements.txt
├── .env
├── knowledge_base/
│   ├── manuals/ (PDF files)
│   └── images/ (PNG/JPG files)
└── chroma_db/
```

9. Expected Output

Problem Identified: Laptop charging issue

Possible Cause: Charger connection or power-delivery issue identified from the retrieved troubleshooting material.

Recommended Steps: (1) Check the charger and power connection. (2) Verify the charging indicator. (3) Try the documented adapter/port checks. (4) Follow the relevant battery or power troubleshooting procedure.

Reference: Troubleshooting Guide + relevant image, when available.

10. Project Outcome

The final system will demonstrate how Naïve RAG can combine heterogeneous technical knowledge—PDF text and visual troubleshooting content—into a single support assistant. The focus is on reliable retrieval, clear source-aware responses, practical troubleshooting steps, and explicit handling of missing knowledge. The project can later be extended with hybrid search, re-ranking, conversation memory, evaluation metrics, OCR, metadata filtering, and multimodal retrieval.